Information Processing Letters 109 (2009) 1066–1070

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

# Finding the longest common nonsuperstring in linear time

Joong Chae Na [a,1], Dong Kyue Kim [b,2], Jeong Seop Sim [c,*]

[a] Department of Computer Science and Engineering, Sejong University, Seoul 143-747, South Korea
[b] Division of Electronics and Computer Engineering, Hanyang University, Seoul 133-791, South Korea
[c] School of Computer and Information Engineering, Inha University, Incheon 402-751, South Korea

ARTICLE INFO

*Article history:*
Received 23 January 2009
Received in revised form 17 June 2009
Accepted 27 June 2009
Available online 1 July 2009
Communicated by M. Yamashita

*Keywords:*
Design of algorithms
String non-inclusion problem
Longest common nonsuperstring
Generalized suffix tree

ABSTRACT

String inclusion and non-inclusion problems have been vigorously studied in such diverse fields as molecular biology, data compression, and computer security. Among the well-known string inclusion or non-inclusion notions, we are interested in the longest common nonsuperstring. Given a set of strings, the longest common nonsuperstring problem is finding the longest string that is not a superstring of any string in the given set. It is known that the longest common nonsuperstring problem is solvable in polynomial time.

In this paper, we propose an efficient algorithm for the longest common nonsuperstring problem. The running time of our algorithm is linear with respect to the sum of the lengths of the strings in the given set, using generalized suffix trees.



## 1. Introduction

String inclusion and non-inclusion problems frequently arise in such diverse fields as data compression [10,11], molecular biology [1,8], and computer security [5]. Examples of well-known inclusion related problems that have been studied vigorously to date [2,3,6] are: the *longest common substring* problem, the *longest common subsequence* problem, the *shortest common superstring* problem, and the *shortest common supersequence* problem. Meanwhile, examples of non-inclusion related problems [4,9] are: the *shortest common nonsubsequence* problem, the *longest common nonsupersequence* problem, the *shortest common nonsubstring* problem, and the *longest common nonsuperstring* problem.

Here, we are interested in the longest common nonsuperstrings. Consider a set of strings $F = \{f_1, \ldots, f_m\}$ and a string $t$ over a constant size alphabet. If $t$ is not a superstring of any $f_i$ for all $1 \leqslant i \leqslant m$, $t$ is called a *common nonsuperstring* of $F$. If $t$ is the longest among all common nonsuperstrings of $F$, $t$ is called the *longest common nonsuperstring* of $F$. For convenience, the common nonsuperstring and the longest common nonsuperstring are denoted as the CNSS and the LCNSS, respectively. The problem of finding the LCNSS is called the *LCNSS problem*. The LCNSS problem was introduced in [12]. Let $S$ denote the set of all the proper suffixes of strings in $F$. Rubinov and Timkovsky [9] defined a directed graph $\Gamma_S = (V, E)$ for $S$ whose path corresponds to a CNSS of $F$. When there is a cycle in $\Gamma_S$, the LCNSS of $F$ does not exist. Otherwise, i.e., when $\Gamma_S$ is a directed acyclic graph, the longest path in $\Gamma_S$, which can be found in $O(|V| + |E|)$ time, represents the LCNSS of $F$. Thus, the remaining part of the solution to the LCNSS problem is how to construct $\Gamma_S$ efficiently. But, no efficient algorithm to construct $\Gamma_S$ has been proposed.

In this paper we propose a linear-time algorithm for finding the LCNSS of $F$. For this, we propose an optimal algorithm for constructing $\Gamma_S$. The main difficulty of constructing $\Gamma_S$ is finding the longest prefix $\alpha$ of a string such that $\alpha$ is in $S \cup F$. We show that the longest prefix $\alpha$ can

* Corresponding author.
*E-mail addresses:* jcna@sejong.ac.kr (J.C. Na), dqkim@hanyang.ac.kr (D.K. Kim), jssim@inha.ac.kr (J.S. Sim).

[1] This work was supported by the Korea Research Foundation (KRF) grant funded by the Korea government (MEST) (No. 2009-0069977).
[2] This work was supported by the Ministry of Knowledge Economy, Korea, under the Information Technology Research Center support program supervised by the Institute of Information Technology Advancement (IITA-2009-C1090-0902-0003).

0020-0190/$ – see front matter © 2009 Elsevier B.V. All rights reserved.
doi:10.1016/j.ipl.2009.06.010

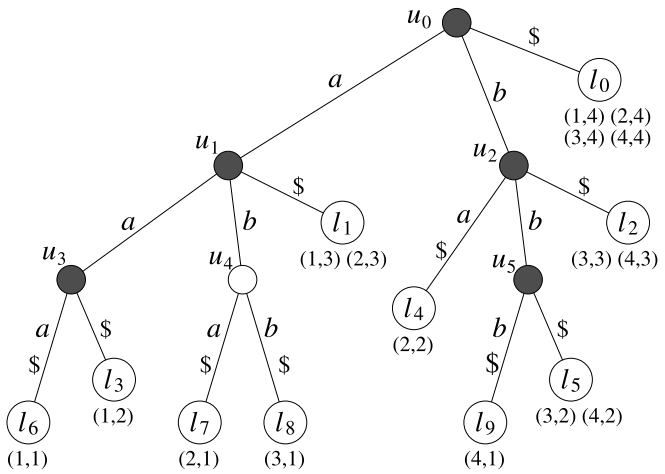



**Fig. 1.** The generalized suffix tree for $F = \{aaa, aba, abb, bbb\}$. Shaded nodes represent suffix-nodes.

be found in constant time, using generalized suffix trees, and thus $\Gamma_S$ can be constructed in time that is linear with respect to the total sum of the lengths of the strings in $F$ for a constant size alphabet.

This paper is organized as follows. In Section 2, we describe the various notations and definitions used in this paper. In Section 3, we explain the graph $\Gamma_S$ that models the LCNSS problem. In Section 4, we describe our algorithm for finding the LCNSS of the given set of strings.

## 2. Preliminaries

Let $\alpha$ be a string over a constant size alphabet $\Sigma$. We denote the length of $\alpha$ by $|\alpha|$ and the $i$th character of string $\alpha$ by $\alpha[i]$. When a string $\beta$ is $\alpha[i]\alpha[i+1]\ldots\alpha[j]$, we denote $\beta$ by $\alpha[i..j]$ and call $\beta$ a *substring* of $\alpha$. Conversely, $\alpha$ is called a *superstring* of $\beta$. A substring of $\alpha$ that ends at position $|\alpha|$ is called a *suffix* of $\alpha$. Moreover, when $i \neq 1$, such suffixes are called *proper suffixes* of $\alpha$. Similarly, we can define a *prefix* of $\alpha$ and a *proper prefix* of $\alpha$, respectively.

We formally define the LCNSS problem.

**Problem 1** *(The longest common nonsuperstring problem (LCNSS problem)).*

**Input:** A set of forbidden strings $F = \{f_1, f_2, \ldots, f_m\}$ over a constant size alphabet $\Sigma$.

**Output:** If no CNSS exists or an arbitrarily long CNSS of $F$ can be made, the output is 'NO'. Otherwise, the output is one of the LCNSSs of $F$.

To avoid some trifling cases, we assume that the LCNSS problem satisfies the following three conditions. Firstly, no $f_i$ is a substring of $f_j$ for $j \neq i$. If $f_i$ is a substring of $f_j$ for some $j \neq i$, any nonsuperstring of $f_i$ is a nonsuperstring of $f_j$. Secondly, no $\sigma$ $(\in \Sigma)$ is an element of $F$. If any $\sigma$ $(\in \Sigma)$ is in $F$, no CNSS of $F$ can contain $\sigma$. Finally, for all $\sigma$ $(\in \Sigma)$, $\sigma^k$ is in $F$ for some $k \geqslant 2$. Otherwise, we can make an arbitrarily long CNSS of $F$ in the form of $\sigma^n$.

To solve the LCNSS problem, we utilize the generalized suffix tree. For each string $f_i \in F$, we define the extended string $f_i'$ as the concatenation of $f_i$ and a special symbol $\$ \notin \Sigma$, i.e., $f_i' = f_i\$$. Let $F' = \{f_1', f_2', \ldots, f_m'\}$. Then, the *generalized suffix tree* $GST_F$ of $F$ is a compacted trie that represents all suffixes of strings in $F'$. See Fig. 1 for an example. We refer the reader to [3] for a formal description. Each edge is labeled with a nonempty string and each leaf represents a suffix of a string in $F'$. $label(u, v)$ denotes the string represented by the edge from node $u$ to node $v$. $L(u)$ denotes the string obtained by concatenating the labels on the path from the root to node $u$. For example, in Fig. 1, $label(u_2, l_4) = a\$$ and $L(u_4) = ab$. Each leaf can represent more than one suffix of a string in $F'$. Thus, each leaf $l$ has a list of pairs of integers, $\langle i, j\rangle$, such that $f_i[j..|f_i|]\$ = L(l)$. These lists are called *suffix lists* and $sl(l)$ denotes the suffix list of a leaf $l$. For example, in Fig. 1, $sl(l_5)$ is $\langle 3, 2\rangle$ and $\langle 4, 2\rangle$ because $f_3[2..3]\$ = f_4[2..3]\$ = bb\$$ $(= L(l_5))$.

## 3. Graph model for the LCNSS problem

In this section we explain the graph model introduced in [9] to solve the LCNSS problem. Recall that $F = \{f_1, \ldots, f_m\}$. Let $\|F\|$ denote the sum of the lengths of the strings in $F$. Let $S$ denote the set of proper suffixes of $f_i$'s $(1 \leqslant i \leqslant m)$. Assume there are $n$ distinct strings in $S$, i.e., $S = \{s_1, \ldots, s_n\}$. Then, a directed graph $\Gamma_S = (V, E)$ is defined over $S$ as follows [9].

1. *the set of vertices* $V$:
   For each $s_i \in S$ $(1 \leqslant i \leqslant n)$, a new vertex $v_i$ is defined and no other vertices exist in $V$. Then, there is a one-to-one correspondence between vertices in $V$ and strings in $S$.
2. *the set of edges* $E$:
   For $s_i \in S$ and $\sigma \in \Sigma$, we define a set $P_i^\sigma$ as follows:

   $$P_i^\sigma = \{y \mid y \in S \cup F \text{ and } y \text{ is a prefix of } \sigma s_i\}.$$

   Assume that $s_j$ $(\in S)$ is an element of $P_i^\sigma$. Then, a directed edge from $v_j$ to $v_i$ is defined if and only if $s_j$ is the longest string in $P_i^\sigma$. This implies that no incoming edge is defined in $v_i$ for $\sigma$ if the longest string in

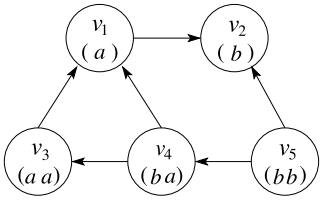



**Fig. 2.** Graph $\Gamma_S$ for $F = \{aaa, aba, abb, bbb\}$. The string in vertex $v_i$ represents the corresponding string $s_i$.

$P_i^\sigma$ is in $F$. Therefore, for each string $s_i \in S$ and character $\sigma \in \Sigma$, at most one edge is defined, and at most $|\Sigma|$ incoming edges are defined for each vertex.

**Example 1.** Let $F = \{aaa, aba, abb, bbb\}$. Then, $S = \{a, b, aa, ba, bb\}$ and $\Gamma_S$ is shown in Fig. 2. Consider a string $s_3$ $(= aa)$. For character $b$, an edge from $v_4$ to $v_3$ is defined since $ba$ $(= s_4)$ is the longest string in $P_3^b = \{b, ba\}$. For character $a$, however, no edge is defined since $aaa$ $(= f_1)$ is the longest string in $P_3^a = \{a, aa, aaa\}$.

When $\Gamma_S$ is defined in the aforementioned manner, Rubinov and Timkovsky [9] proved Lemma 1, which shows the relationship between a path in $\Gamma_S$ and a CNSS of $F$.

**Lemma 1.** *(See [9].) If $\Gamma_S$ is a directed acyclic graph (DAG), the longest path of $\Gamma_S$ corresponds to the LCNSS of $F$. When the longest path has been found, the LCNSS can be obtained as follows*: (i) *Concatenate the first character of the corresponding string of each vertex in the longest path before the final vertex and,* (ii) *concatenate the complete string corresponding to the final vertex of the path.*

**Example 2.** Consider set $F$ and $\Gamma_S$ in Example 1. The longest path in $\Gamma_S$ is $\langle v_5, v_4, v_3, v_1, v_2\rangle$ and thus, the LCNSS for $F = \{aaa, aba, abb, bbb\}$ is $bbaab$.

## 4. Algorithm

### 4.1. Outline

The proposed algorithm for the LCNSS problem consists of three phases. In the first phase, we make the generalized suffix tree $GST_F$ for $F$. Recall that $F = \{f_1, \ldots, f_m\}$ and $S = \{s_1, \ldots, s_n\}$. Then, $GST_F$ has $m+n+1$ leaves. (Note that $m$ leaves correspond to $m$ strings in $F$, $n$ leaves correspond to $n$ strings in $S$, and one leaf represents the empty string $\epsilon$.) Let $l_i$ $(0 \leqslant i \leqslant m+n)$ denote the leaf node of $GST_F$ and $str(l_i)$ denote the string obtained by removing \$ at the end of $L(l_i)$. For convenience, we assume that $l_0$ is the leaf such that $L(l_0) = \$$. Then $str(l_0)$ represents $\epsilon$. Also, we assume that $|str(l_i)| \leqslant |str(l_j)|$ for $0 \leqslant i \leqslant j \leqslant m+n$. (We can sort the suffixes of $F$ according to their lengths, in nondecreasing order, using a linear-time sorting algorithm.) Since $GST_F$ can be constructed in $O(\|F\|)$ time [7,13,3], the first phase runs in $O(\|F\|)$ time.

In the second phase, we make $\Gamma_S$ using $GST_F$. This is the main contribution of this paper. We describe this phase in the next subsection. In the final phase, we first find the longest path (if it exists) in $\Gamma_S$ in $O(|V| + |E|)$ time. Then, we construct the LCNSS by traversing each vertex in the longest path, in the manner explained in the previous section. Since $\Gamma_S$ has $n$ $(\in O(\|F\|))$ vertices and at most $n|\Sigma|$ edges, the final phase also runs in $O(\|F\|)$ time for a constant size alphabet.

### 4.2. Constructing $\Gamma_S$

We first give the various definitions and notations used in $GST_F$. A node $x$ is called a *suffix-node* if $L(x) \in S \cup F$. In $GST_F$, $x$ is a suffix-node if it has a child (leaf) $y$ such that $label(x, y) = \$$. The leaf $y$ is called the *suffix-leaf* of $x$. In the example of Fig. 1, all internal nodes except for $u_4$ are suffix-nodes. The *nearest suffix-ancestor* of a node $x$ (denoted by $nsa(x)$) is defined as the nearest ancestor $z$ of $x$ such that $z$ is a suffix-node. (Note that $z$ can equal $x$.)

**Lemma 2.** *Let $l$ be a leaf in $GST_F$ and $w$ be the nearest suffix-ancestor of $l$ such that $L(w) \neq str(l)$. Then, $L(w)$ is the longest proper prefix of $str(l)$ in $S \cup F$.*

**Proof.** It is clear that $L(w)$ is a proper prefix of $str(l)$ in $S \cup F$. Then, we show by contradiction that $L(w)$ is the longest one. Suppose that there is a prefix $\delta$ of $str(l)$ such that $\delta \in S \cup F$ and $|L(w)| < |\delta| < |str(l)|$. Since $GST_F$ represents all strings in $S \cup F$, there exists a leaf $l'$ such that $L(l') = \delta\$$. Also, there exists a common ancestor $w'$ of $l$ and $l'$ such that $L(w') = \delta$ and $label(w', l') = \$$, which implies that $w'$ is a suffix-node. Then, $w'$ must exist between $w$ and $l$, which contradicts the definition of $w$. □

For a leaf $l_i$ and a character $\sigma$, we define the link function $link(l_i, \sigma)$ as follows: Let $\alpha$ be the longest prefix of $\sigma str(l_i)$ such that $\alpha \in S \cup F$. Since leaves of $GST_F$ represent strings in $S \cup F$, there exists a leaf $l_j$ such that $str(l_j) = \alpha$. Then, $link(l_i, \sigma)$ is defined as leaf $l_j$.

The second phase consists of three stages.

1. Compute $nsa(x)$ for every node $x$.
2. Compute $link(l_i, \sigma)$ for every leaf $l_i$ and character $\sigma$.
3. Construct $\Gamma_S$ using the *link* function.

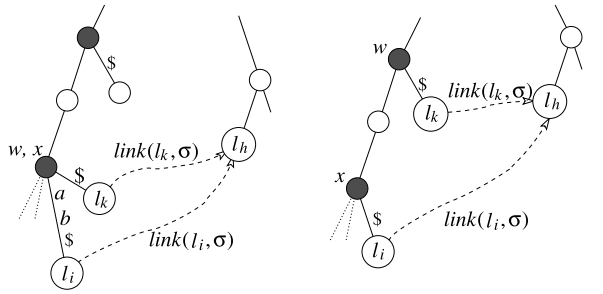
#### 4.2.1. Stage 1

For each node $x$ in the preorder traversal, we compute $nsa(x)$. If $x$ is a suffix node, we set $nsa(x) = x$; otherwise, we set $nsa(x) = nsa(parent(x))$. Note that the root is a suffix node. Since it takes constant time to check whether $x$ is a suffix node or not, we can compute $nsa(x)$ for every node $x$ in $O(\|F\|)$ time.

#### 4.2.2. Stage 2

We compute $link(l_i, \sigma)$ for every leaf $l_i$ and character $\sigma$ in order of increasing $i$. Initially we compute $link(l_0, \sigma)$ for each character $\sigma$. Recall that $l_0$ is the suffix-leaf of the root. Let $z$ be the leaf such that $L(z) = \sigma\$$. Note that, because $\sigma^k \in F$ $(k \geqslant 2)$, $z$ must exist in $GST_F$, which can be found in constant time. Since $str(l_0)$ is $\epsilon$, $\sigma$ is the longest prefix of $\sigma str(l_0)$ in $S \cup F$. Thus, we set $link(l_0, \sigma) = z$.

Then, we show how to compute $link(l_i, \sigma)$ for every leaf $l_i$ $(i \geqslant 1)$ and character $\sigma \in \Sigma$. This consists of two parts.




(a) Case when $label(x, l_i) \neq \$$ (b) Case when $label(x, l_i) = \$$

**Fig. 3.** Part 2 of Stage 2. Shaded nodes represent suffix-nodes.

| $\sigma \diagdown i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| $a$ | $l_1$ | $l_3$ | $l_1$ | $l_6$ | $l_7$ | $l_8$ | $l_6$ | $l_3$ | $l_3$ | $l_8$ |
| $b$ | $l_2$ | $l_4$ | $l_5$ | $l_4$ | $l_5$ | $l_9$ | $l_4$ | $l_4$ | $l_4$ | $l_9$ |

**Fig. 4.** Function $link(l_i, \sigma)$ of $GST_F$ for $F = \{aaa, aba, abb, bbb\}$.

**Part 1.** Compute $link(l_i, \sigma)$ for characters $\sigma$ such that $\sigma str(l_i) \in S \cup F$.

For this, we use the suffix list $sl(i)$. For each element $\langle p, q\rangle \in sl(i)$ such that $q \neq 1$, let $l_j$ be the leaf such that $str(l_j) = f_p[q-1..|f_p|]$. Given $p$ and $q-1$, $l_j$ can be found in constant time. Then we set $link(l_i, f_p[q-1]) = l_j$.

**Part 2.** Compute $link(l_i, \sigma)$ for characters $\sigma$ such that $\sigma str(l_i) \notin S \cup F$.

For this, we use the *link* function of another leaf. Let $\alpha$ be the longest proper prefix of $str(l_i)$ in $S \cup F$. We first find the suffix-node $w$ such that $L(w) = \alpha$. By Lemma 2, $w$ is the nearest suffix-ancestor of $l_i$ such that $L(w) \neq str(l_i)$. Let $x$ be the parent node of $l_i$. Then, $w = nsa(x)$ if $label(x, l_i) \neq \$$, otherwise $w = nsa(parent(x))$. (See Fig. 3.) Let $l_k$ be the suffix-leaf of $w$. Then, $link(l_i, \sigma) = link(l_k, \sigma)$, which is proven in Lemma 3.

We show various examples of computing the *link* function for $GST_F$ of Fig. 1. The complete results of the *link* function are shown in Fig. 4.

**Example 3.** Consider leaf $l_5$ in Fig. 1. This example shows the case when $\sigma str(l_i) \in S \cup F$ for every character $\sigma$ (Part 1). There are two elements $\langle 3, 2\rangle$ and $\langle 4, 2\rangle$ in the suffix list $sl(5)$. Since pair $\langle 3, 1\rangle$ is an element of $sl(8)$ and $f_3[1] = a$, we obtain $link(l_5, a) = l_8$. Similarly, for pair $\langle 4, 2\rangle$, we obtain $link(l_5, b) = l_9$.

**Example 4.** Consider leaf $l_2$ in Fig. 1. First, we process elements $\langle 3, 3\rangle$ and $\langle 4, 3\rangle$ in $sl(2)$. From these elements, we obtain $link(l_2, b) = l_5$. Because Part 1 cannot compute $link(l_2, a)$, this is computed in Part 2. Since $label(u_2, l_2) = \$$, we check suffix-leaf $l_0$ of $nsa(parent(u_2))$ $(= u_0)$ and get $link(l_2, a) = link(l_0, a) = l_1$.

**Example 5.** Consider leaf $l_9$ in Fig. 1. There is no pair in $sl(9)$ for which the second element is not 1. Thus, no link function of $l_9$ can be computed in Part 1. Since $label(u_5, l_9) \neq \$$, we check suffix-leaf $l_5$ of $nsa(u_5)$ $(= u_5)$ and obtain $link(l_9, \sigma) = link(l_5, \sigma)$ for every $\sigma$, that is, $link(l_9, a) = l_8$ and $link(l_9, b) = l_9$.

**Lemma 3.** *Stage 2 computes the link function correctly in $O(\|F\|)$ time.*

**Proof.** We first consider the correctness of Stage 2. It is clear that the *link* function of $l_0$ is computed correctly. For $i \geqslant 1$, there are two cases for a character $\sigma \in \Sigma$.

1. Case that $\sigma str(l_i) \in S \cup F$.
2. Case that $\sigma str(l_i) \notin S \cup F$.

The first and second cases are handled in Parts 1 and 2, respectively. First, consider the first case, where $\sigma str(l_i)$ is a string $(f_p[q-1..|f_p|])$ in $S \cup F$. Then, pair $\langle p, q\rangle \in sl(i)$ since $f_p[q..|f_p|] = str(l_i)$. Thus, by the definition, $link(l_i, \sigma)$ is the leaf $l_j$ such that $str(l_j) = f_p[q-1..|f_p|]$. Note that $q$ cannot be 1. Moreover, it is clear that Part 1 handles only the first case.

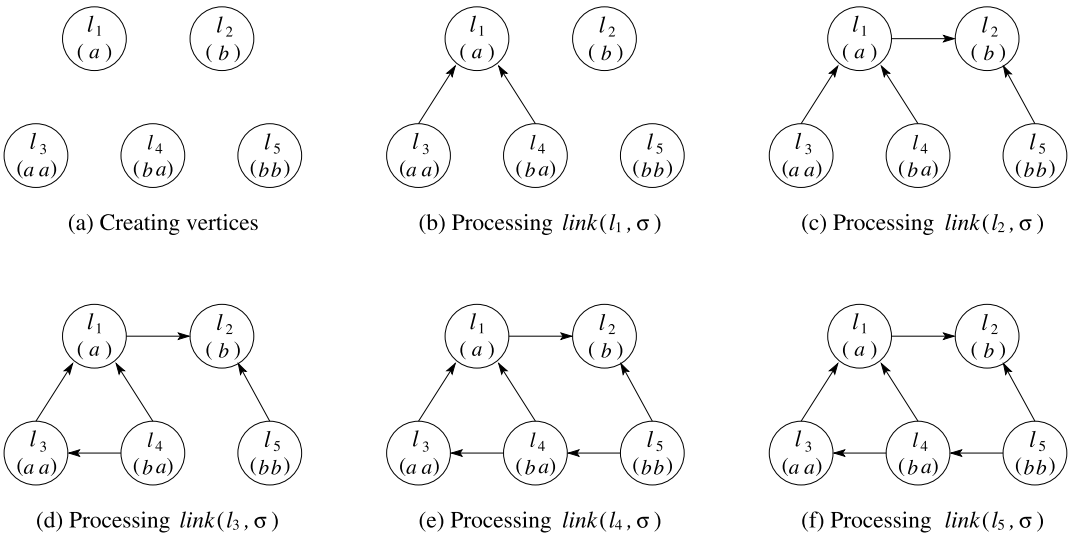
Next, consider the second case. Let $\beta = str(l_i)$. Recall that $\alpha$ is the longest proper prefix of $\beta$ in $S \cup F$ and $\alpha = L(w) = str(l_k)$. Let $l_h = link(l_k, \sigma)$ and $\sigma\delta = str(l_h)$. By the definition of the *link* function, $\sigma\delta$ is the longest prefix of $\sigma\alpha$ in $S \cup F$. Now we show by contradiction that $\sigma\delta$ is also the longest prefix of $\sigma\beta$ in $S \cup F$, i.e., $link(l_i, \sigma) = l_h$. Suppose that there exists a prefix $\sigma\delta'$ of $\sigma\beta$ in $S \cup F$ such that $|\delta'| > |\delta|$. If $|\delta'| \leqslant |\alpha|$, $\sigma\delta'$ is a prefix of $\sigma\alpha$ and this contradicts the fact that $\sigma\delta$ is the longest prefix of $\sigma\alpha$ in $S \cup F$. If $|\delta'| > |\alpha|$, this contradicts the fact that $\alpha$ is the longest prefix of $\beta$ in $S \cup F$. Thus, such $\delta'$ does not exist.

Now we consider the time complexity. Part 1 can be done in constant time for each element of a suffix list. Since the total number of elements in all suffix lists is at most $\|F\|$, Part 1 can be done in $O(\|F\|)$ time for all leaves. In Part 2, $l_k$ can be found in constant time and the *link* function of $l_k$ has already been computed, because $k < i$. Thus, Part 2 also can be done in constant time for each leaf $l_i$. Therefore, Stage 2 can be done in $O(\|F\|)$ time overall. □

*4.2.3. Stage 3*

$\Gamma_S$ is constructed in $O(\|F\|)$ time using $GST_F$ and the *link* function. First, we create vertices of $\Gamma_S$ using leaves of $GST_F$. For each leaf $l_b$ of $GST_F$ $(b \geqslant 1)$, the following is done: If $str(l_b)$ is a string $(s_j)$ in $S$, a vertex labeled with $v_j$ is created (we do not need to know index $j$); otherwise, nothing is done. Note that $str(l_b)$ is in either $S$ or $F$, which can be checked in constant time because $str(l_b)$ is in $F$ if and only if $sl(b)$ has only one pair $\langle p, q\rangle$ and $q = 1$.

Next, the edges of $\Gamma_S$ are created using the *link* function. Consider a leaf $l_b$ of $GST_F$ such that $str(l_b)$ is a string $(s_j)$ in $S$. For each character $\sigma$, set $l_a = link(l_b, \sigma)$. Recall that $str(l_a)$ is the longest prefix of $\sigma s_j$ in $S \cup F$ by the definition of the *link* function. According to the definition of $\Gamma_S$, if $str(l_a)$ is a string $(s_i)$ in $S$, an edge from $v_i$ to $v_j$ is created; otherwise, i.e., $str(l_a)$ is a string in $F$, nothing is done.




(a) Creating vertices (b) Processing $link(l_1, \sigma)$ (c) Processing $link(l_2, \sigma)$

(d) Processing $link(l_3, \sigma)$ (e) Processing $link(l_4, \sigma)$ (f) Processing $link(l_5, \sigma)$

**Fig. 5.** Constructing $\Gamma_S$ for $F = \{aaa, aba, abb, bbb\}$. Vertices are labeled with their corresponding leaves.

**Example 6.** Consider $GST_F$ in Fig. 1 and its link function in Fig. 4. We first create vertices of $\Gamma_S$ as in Fig. 5(a). Only for $1 \leqslant b \leqslant 5$, we create a vertex representing $str(l_b)$ because $str(l_b) \in S$. Next, we create edges using the *link* function. Figs. 5(b)–5(f) show the graphs after processing the *link* function for each leaf from $l_1$ to $l_5$. Consider leaf $l_3$. Because $link(l_3, b) = l_4$ and $str(l_4) \in S$, we create an edge from the vertex representing $str(l_4)$ to the vertex representing $str(l_3)$. However, we create no edge for $link(l_3, a)$ because $link(l_3, a) = l_7$ and $str(l_7) \in F$.

**Lemma 4.** *Stage* 3 *constructs* $\Gamma_S$ *correctly in* $O(\|F\|)$ *time.*

**Proof.** It is clear that Stage 3 constructs a subgraph of $\Gamma_S$.

We show that all vertices and edges of $\Gamma_S$ are created by Stage 3. For every vertex $v_j$ of $\Gamma_S$, its corresponding string $s_j$ is an element in $S$. Since leaves of $GST_F$ represent all strings in $S \cup F$, there must be a leaf $l_b$ of $GST_F$ such that $str(l_b) = s_j$. Therefore, vertex $v_j$ is created by Stage 3.

For every edge $\langle v_i, v_j \rangle$ of $\Gamma_S$, let $l_a$ and $l_b$ be the leaves of $GST_F$ such that $str(l_a) = s_i$ and $str(l_b) = s_j$, respectively. Consider $link(l_b, \sigma)$, where $\sigma$ is the first character of $str(l_a)$. By the definition of the link function, $str(l_a)$ is the longest prefix of $\sigma\, str(l_b)$ in $S \cup F$. Since $str(l_a) = s_i$ is a string in $S$, edge $\langle v_i, v_j \rangle$ is created by Stage 3.

It is clear that Stage 3 can be done in constant time for each leaf of $GST_F$. Thus, Stage 3 can be done in $O(\|F\|)$ time overall. □

**Theorem 1.** *The LCNSS problem is solvable in linear time, i.e.,* $O(\|F\|)$ *time.*

## References

[1] R. Drmanac, C. Crkvenjakov, Sequencing by hybridization (SBH) with oligonucleotide probes as an integral approach for the analysis of complex genomes, International Journal of Genomic Research 1 (1) (1992) 59–79.
[2] M.R. Garey, D.S. Johnson, Computers and Intractobility, W.H. Freeman and Company, 1979.
[3] D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge Univ. Press, 1997.
[4] T. Jiang, V.G. Timkovsky, Shortest consistent superstrings computable in polynomial time, Theoretical Computer Science 143 (1) (1995) 113–122.
[5] F. Kerschbaum, A new way to think about secure, in: Computation: Language-based Secure Computation, Proc. 5th Int. Workshop on Security in Information Systems, 2007, pp. 33–42.
[6] D. Maier, The complexity of some problems on subsequences and supersequences, Journal of ACM 25 (1978) 322–336.
[7] E.M. McCreight, A space-economical suffix tree construction algorithm, Journal of ACM 23 (1976) 262–272.
[8] P. Pevzner, R. Lipshutz, Towards DNA sequencing by hybridization, Manuscript, 1993.
[9] A.R. Rubinov, V.G. Timkovsky, String noninclusion optimization problems, SIAM J. Discrete Mathematics 11 (3) (1998) 456–457.
[10] J. Store, T. Szymanski, Data compression via textual substitution, Journal of ACM 29 (1982) 928–961.
[11] J. Storer, Data Compression: Methods and Theory, Computer Science Press, 1988.
[12] V.G. Timkovsky, Complexity of common subsequence and supersequence problems and related problems, Kibernetika 5 (1989) 1–13.
[13] E. Ukkonen, On-line construction of suffix trees, Algorithmica 14 (3) (1995) 249–260.